Seminar 3

Object-Oriented Design, IV1350

Mo Wang mowang@ug.kth.se

2026-04-18

Project Members:
[Mo Wang, mowang@ug.kth.se]

Declaration:

By submitting this assignment, it is hereby declared that all group members listed above have contributed to the solution. It is also declared that all project members fully understand all parts of the final solution and can explain it upon request.

It is furthermore declared that no part of the solution has been copied from any other source (except for resources presented in the Canvas page for the course IV1350), and that no part of the solution has been provided by someone not listed as a project member above.

1 Introduction

This report delves into the implementation and testing of the *Repair Electric Bike* system. The purpose of Seminar 3 is to translate the design produced in Seminar 2 into executable source code and to verify the program behavior using unit tests. The implementation process follows the guidelines of object-oriented programming and testing, which are described in Chapters 6 and 7 of *A First Course in Object-Oriented Development* [1].

The implementation follows the previously designed class diagrams and communication diagrams as an initial structure. During implementation, additional decisions regarding encapsulation, cohesion, coupling, naming, and code structure are continuously made in order to produce readable, maintainable, and robust Java code.

Automated unit testing is carried out in parallel with the implementation that goes parallel to the implementation. Tests are implemented using JUnit to validate the behavior of the core parts of the system and to ensure that all parts of the program work as intended when changes are made. The tests are designed according to the testing principles presented in Chapter 7 of the course book, emphasizing self-evaluating tests and meaningful coverage of program behavior [1].

1.1 Requirements of Seminar 3

Seminar 3 consists of two main tasks: program implementation and unit testing. Unit tests are carried out incrementally after implementing each system operation.

The first task is to implement the basic flow and the Startup sequence of the Repair Electric Bike system that was designed in Seminar 2. The implementation follows these requirements:

- The program is written in Java and is compilable and executable.
- Only the basic workflow and startup sequence are implemented while alternative flows are intentionally omitted in this seminar.
- The user interface is replaced by a single `View` class containing hard-coded calls to the controller, which prints all returned results to `System.out`, including the repair order printout.
- No database or external systems are implemented. Instead, the integration layer stores data in memory using placeholder objects.
- Exceptions are not used, since exception handling is introduced in a later seminar.
- The code follows the programming guidelines described in Chapter 6 of the course book, including naming conventions, high cohesion, low coupling, good encapsulation, and one Javadoc comment for each public declaration.

The second task is to write unit tests. The testing requirements are as follows:

- Unit tests are implemented using the JUnit framework.
- Tests cover all relevant classes in the controller, model, and integration layers.
- Simple DTO classes, as well as methods consisting only of constructors, setters, and getters, are excluded from testing.
- Methods that produce output to `System.out` are not tested, in accordance with the seminar requirements.

1.2 Implementing Code Based on Design Diagrams

The class diagrams and communication diagrams of system operations from Seminar 2 build the foundation for implementing the program structure and behavior. The class diagram provides a blueprint for the overall architecture and package structure, while the communication diagrams provide guidance for implementing of system operations by explicitly describing message flow and object interaction.

By following these diagrams during implementation, the intended execution flow of each system operation is preserved and ambiguity is reduced. At the same time, the implementation is continuously evaluated against object-oriented design principles to ensure that the resulting code remains cohesive, encapsulated, and loosely coupled.

2 Method

This chapter describes the development process for implementing and testing the *Repair Electric Bike* system in Seminar 3. It explains how the design diagrams were translated into executable Java code, how unit tests were carried out, and which tools and guidelines were used.

2.1 Implementation Approach

The implementation process started by translating the design diagrams produced in Seminar 2 into an initial Java code structure. The class diagram was used to define the overall architecture, including package structure and class responsibilities, while the communication diagrams guided the incremental implementation of each system operation.

Development began by creating package structures and class skeletons without functionality, according to the class diagrams. These skeletons were then extended iteratively, implementing one system operation at a time. Special attention was given to the startup sequence to ensure that all necessary objects were created and connected correctly before executing system operations. After each system operation, tests were written to detect potential errors, which were resolved before continuing with the next system operation.

Throughout the implementation, object-oriented programming guidelines from Chapter 6 of the course book were continuously applied. Design decisions regarding encapsulation, cohesion, and coupling were revisited when needed, and minor design adjustments were made whenever implementation revealed potential improvements.

2.2 Dividing Code into Packages

The code was organized into packages corresponding to the layered MVC architecture defined during design. Separate packages were created for the controller, model, integration, and startup layers, each with a clearly defined responsibility.

Classes that did not naturally belong to a specific layer, such as utility functionality used across multiple layers, were placed in a separate `util` package. This organization improved readability and helped maintain low coupling between layers while preserving high cohesion within each package.

The separation into layers produced low coupling and high cohesion, as well as improved readability, maintainability, and extensibility, since each package has a well-defined responsibility.

2.3 Following code conventions

During implementation, Java code conventions were followed consistently to improve readability and maintainability. Package names are written in lowercase and structured to reflect project organization, while class and interface names use descriptive identifiers written in `PascalCase`.

Method, field, and variable names follow `camelCase` naming conventions and are chosen to clearly describe their purpose. Constants declared as `static final` fields are written in uppercase letters with words separated by underscores. Adhering to these conventions ensures that code intent is clear and reduces ambiguity during further development and review.

2.4 Writing comments and Javadoc

Documentation of the public interfaces was handled using Javadoc comments, following the recommendations in Chapter 6 of the course book, in order to generate clear Java API documentation. Each public class and method includes a Javadoc description explaining its purpose, parameters, return values, and possible exceptions at a high level, without exposing internal implementation details.

Inline comments inside methods were deliberately avoided, since they tend to focus on internal implementation details rather than overall intent. Instead, clarity was achieved by using well-named methods and variables, allowing implementation details to remain encapsulated and easily modifiable without changing public interfaces.

2.5 Code smell and refactoring

During development, code smells such as duplicated code, long methods, and excessive use of primitive values were identified. When such issues were identified, the code was refactored to improve readability, cohesion, and maintainability without changing program behavior.

Refactoring was performed incrementally after implementing each system operation. This continuous improvement process helped ensure that the code remained clean and aligned with object-oriented design principles as the program evolved.

One common code smell is duplicated code, where similar logic appears in different methods. This can be resolved by extracting the shared functionality logic into separate helper methods, some are private methods if these are not used elsewhere. For instance, the formatting styles are common between the class `RepairOrder` and `RepairOrderDTO`, whereas the `format` function, that is only responsible for formatting the repair order string, can be extracted and placed in `RepairOrderDTO` class.

Another common code smell is long methods, which can be defined as a method taking responsibility of more than one functionality without explicit hierarchy of method calling. This can be resolved by splitting up each logic of the method that concerns each responsibility into its intended well-named method. This improves readability and makes the control flow easier to follow and understand.

Additionally, primitive obsession is another code smell, where primitive values are favoured over objects. This can be addressed by introducing small data classes, DTOs, that wrap up multiple variables into one, such as `Cost` class that holds both an amount value and a currency value. This avoids primitive variables and fields to be scattered across the logic, which does not only cause low readability, but also harder to maintain

since multiple variables have to be considered without an explicit relationship written as code syntax.

2.6 Testing Strategy

Unit testing was integrated into the development process and performed iteratively throughout the implementation, following the testing principles described in Chapter 7 of *A First Course in Object-Oriented Development*. Testing was not treated as a separate final activity but as an integral part of each iteration. After completing a system operation, corresponding unit tests were written and executed before continuing with further development.

Isolation and correctness The primary goal of the unit tests is to verify functional correctness in isolation. Each test focuses on a single unit of behavior and avoids dependencies on external systems or user interfaces. In accordance with the seminar requirements and Chapter 7 guidelines, methods that produce output to `System.out` are not tested, and tests are instead concentrated on the controller, model, and integration layers where business logic and coordination occur.

Boundary conditions Test cases were selected to cover both typical execution paths and boundary cases. For example, lookup operations are tested with valid input to verify expected behavior, as well as with invalid or non-existing identifiers to verify that the system reacts correctly in error situations. This ensures that alternative branches in conditional logic are exercised, which increases confidence in the correctness of the implementation.

Self-evaluating tests All tests are self-evaluating, meaning that a test produces an informative failure message when it fails and produces no output when it passes, as recommended in Chapter 7. This makes it possible to verify program behavioral correctness by running the test suite with a single command and immediately identifying failing behavior. The tests thereby also function as regression tests, ensuring that previously working functionality remains intact when the code is modified, extended, or refactored.

Test structure Tests are structured so that each test class corresponds to exactly one production class, and test classes are placed in the same package as the system under test but in a parallel test source directory. Simple data transfer objects (DTOs), as well as methods containing only constructors, getters, or setters, are excluded from testing, since they contain no logic to verify and including them would add little value.

Incremental development quality This testing strategy supports incremental development by detecting faults early, reducing the risk of introducing regressions, and increasing confidence that the program behaves as intended at each stage of implementation.

3 Result

Based on the design diagrams from Seminar 2, the code implementation covers the basic flow and startup sequence. Additionally, the code implementation is structurally divided into multiple packages, where each of them corresponds to a layer in the MVC-layered architecture.

Each system operation is described in terms of its purpose and execution flow through the different layers of the system. Relevant parts of the implementation are illustrated with code snippets, and the interaction between the controller, model, and integration layers is explained.

The complete source code for this seminar is available in the public Git repository at: `https://github.com/mo-mosverkstad/java-learning/tree/main/maven-projects/ElectricBikeRepa` [2].

3.1 Implementation results

3.1.1 `startup` sequence

The startup sequence involves instantiation of the necessary entity class objects of the program, which can be mapped to the communication diagram of the start sequence. The sequence is important during development as the first to implement, since otherwise the program does not have a clear starting point and can be harder to track as the code grows large. Additionally, it also provides a possibility for manually checking program behavior during implementation.

The startup sequence initializes the View, Controller, and Registry classes, which together form the core structure of the application. At the top of the layered MVC architecture, the View class provides a simplified interface with hard-coded input and output to the command line. Since there are two different roles, two separate classes `ReceptionistController` and `TechnicianController` are created, according to the design. However, some of their authorization actions overlap, both classes share a common superclass `Controller`. Both controllers are initialized and handled collectively by a `ControllerCreator` class. In this case, only the creator class needs to be initialized instead of separately initializing each class. As the number of roles and controllers grows, bundling them together becomes easier to extend and maintain than keeping them as separate classes. In integration layer, the registry classes handle the data registry. Since the code does not handle databases, the implementation will store the data in-memory instead. The registry classes consist of `CustomerRegistry` and `RepairOrderRegistry`. `CustomerRegistry` manages customer data and detailed information, while `RepairOrderRegistry` manages repair orders as well as their details. Both registry classes are collectively handled by a `RegistryCreator`, which is a class that manages both registry classes, with the same pattern as for handling controllers.

In this case, the fundamental functional skeleton of the program can be placed out, whereas each system operation can easily be added by extending the existing structure.

This snippet shows how the application is started and how the different layers are connected. The startup logic is centralized and contains no business logic, which keeps

object creation separated from domain behavior.

The main class initializes all the class object and acts as the application entry point. It establishes the overall program structure by creating instances of the integration, controller, and view layers, while connecting them together as a starting point integration.

Listing 1: Main class and main method entry point

```java
public class Main {
    public static void main(String[] args) {
        RegistryCreator registryCreator = new RegistryCreator();
        Printer printer = new Printer();
        ControllerCreator controllerCreator = new ControllerCreator(
    registryCreator, printer);
        View view = new View(controllerCreator);
        view.proceedActions("0707654321", "SK-2024-055");
    }
}
```

The View class currently contains a dummy view interface with hardcoded input as well as output to command line interface by using system print.

Actions proceeded by receptionist and technician are hardcoded and collected in the method `proceedActions` that calls the internal private functions. Internal functions `proceedReceptionPreparationActions` and `proceedReceptionConfirmationActions` are proceeded by receptionist using `ReceptionistController` while `proceedTechnicianDiagnosticActio` is performed by the technician using the `TechnicianController`. Each operation may yield exceptions, which can be caused caused by the business logic errors, such as entering a non-existent telephone number. In such case, a try catch block is surronded around the private internal methods where both receptionist and technician actions are performed, while the exceptions are caught and printed to the view with a simple message.

Listing 2: Skeleton code of `View` class

```java
package se.ebikerepair.view;

/**
 * View layer that orchestrates the repair workflow: reception preparation,
    technician diagnostics,
 * and reception confirmation. Catches and displays user-facing errors.
 */
public class View {
    private static final String ERROR_PREFIX = "Error: ";
    private final ReceptionistController receptionistController;
    private final TechnicianController technicianController;

    /**
     * Creates a view with controllers from the given controller creator.
     *
     * @param controllerCreator the factory providing access to controllers
     */
    public View(ControllerCreator controllerCreator) {
```

```java
        receptionistController = controllerCreator.getReceptionistController();
        technicianController = controllerCreator.getTechnicianController();
    }

    /**
     * Runs the full repair workflow: reception preparation, technician diagnostics, and reception confirmation.
     *
     * @param telephoneNumber the customer's telephone number
     * @param bikeSerialNumber the serial number of the broken bike
     */
    public void proceedActions(String telephoneNumber, String bikeSerialNumber) {
        try{
            proceedReceptionPreparationActions(telephoneNumber, bikeSerialNumber);
            proceedTechnicianDiagnosticActions(telephoneNumber);
            proceedReceptionConfirmationActions(telephoneNumber);
        } catch (NonExistentTelephoneNumberException | InvalidTelephoneNumberException | FailedOperationException | IllegalArgumentException | IllegalStateException e) {
            System.out.println(ERROR_PREFIX + e.getMessage());
        }
    }
    ...
}
```

3.1.2 The `searchCustomer` system operation

The first implemented system operation is `searchCustomer`, which involves retrieving customer information based on a provided telephone number. This operation is initiated from the View layer and handled by the `ReceptionistController`, following the interaction defined in the communication diagram.

The execution of this operation begins when the View calls `searchCustomer(telephoneNumber)` in the controller. The controller normalizes the input telephone number into E.164 format using the `TelephoneNumber` utility class. The number is used as a key to find the customer in `CustomerRegistry`, which returns the corresponding `CustomerDTO`, or `null` if no match is found. In the end, the controller validates the result: If a customer is found, it is returned to the caller. Otherwise, an `IllegalArgumentException` is thrown.

This applies the separation of concerns, where the input normalization is handled by a utility class, outside of the main logic. Additionally, data access is handled by the registry, while the control flow and validation are handled by the controller. In terms of encapsulation, data structures of registry are hidden from controller.

This code illustrates how input normalization, data access, and validation responsibilities are clearly separated. The controller coordinates the operation, while telephone formatting and data storage are delegated to specialized classes.

Listing 3: Searching a customer using `searchCustomer` method in `ReceptionistController`

```java
/**
 * Controller handling receptionist operations: customer search, repair order creation, acceptance and rejection.
 */
public class ReceptionistController extends Controller{
    private final CustomerRegistry customerRegistry;
    private final Printer printer;

    /**
     * Creates a receptionist controller with access to registries and a printer.
     *
     * @param registryCreator the creator providing access to data registries
     * @param printer the printer for printing accepted repair orders
     */
    public ReceptionistController(RegistryCreator registryCreator, Printer printer){
        super(registryCreator.getRepairOrderRegistry());
        customerRegistry = registryCreator.getCustomerRegistry();
        this.printer = printer;
    }

    /**
     * Searches for a customer by telephone number.
     *
     * @param telephoneNumber the customer's telephone number (any Swedish format)
     * @return the found customer DTO
     * @throws NonExistentTelephoneNumberException if the phone number does not correspond to a customer
     * @throws InvalidTelephoneNumberException if the telephone number is empty or invalid
     */
    public CustomerDTO searchCustomer(String telephoneNumber) throws NonExistentTelephoneNumberException, InvalidTelephoneNumberException, FailedOperationException{
        try {
            String phoneNumberE164 = new TelephoneNumber(telephoneNumber).toE164();
            return customerRegistry.find(phoneNumberE164);
        } catch (ResourceNotFoundException | InvalidResourceURIException | CannotReadFileException | CannotWriteFileException | NoDatabaseException exception){
            throw new FailedOperationException("Fail to search customer", exception);
        }
    }
```

```
}
```

3.1.3 The `createRepairOrder` system operation

The `createRepairOrder` operation is responsible for creating a new repair order for a specific customer and their bike. The operation is initiated by the View after a customer has been successfully retrieved. The View provides the telephone number and a `ProblemDTO`, which contains both the bike and a description of the issue.

To start with, the View calls `createRepairOrder` in the `ReceptionistController`. The controller retrieves the customer using the telephone number, as a new `RepairOrder` object is created in the model layer. The problem description is added to the repair order, which is stored in the `RepairOrderRegistry`. In the end, the unique repair order ID is returned back to the View

This operation shows how responsibilities are distributed across layers. The controller coordinates the process while the model encapsulates business logic, and the registry manages persistence. By generating a unique id for each repair order, the system ensures traceability and enables future system operations such as diagnostics and task management.

As a result, a new repair order is successfully created and stored, allowing the workflow to proceed to the diagnostic phase.

Listing 4: Method `proceedReceptionPreparationActions` in `View` for proceeding receptionist preparation tasks

```java
private void proceedReceptionPreparationActions(String telephoneNumber, String
    bikeSerialNumber) throws NonExistentTelephoneNumberException,
    InvalidTelephoneNumberException, FailedOperationException {
    CustomerDTO foundCustomer = receptionistController.searchCustomer(
    telephoneNumber);
    printout("1. Reception - Found customer:", foundCustomer);

    BikeDTO foundBike = foundCustomer.getBikeBySerialNumber(bikeSerialNumber);
    String repairOrderId = receptionistController.createRepairOrder(
    telephoneNumber, new ProblemDTO("Broken bike chain", foundBike));
    printout("2. Reception - Created repair order with id: ", repairOrderId);
}
```

The detailed implementation of `createRepairOrder` starts by finding a customer by a given telephone number, while a `RepairOrder` instance is created. The repair order is then saved into the `repairOrderRegistry`.

Listing 5: Creating a new repair order using `createRepairOrder` method in `ReceptionistController`

```java
/**
 * Creates a new repair order for the customer identified by telephone number.
 *
 * @param telephoneNumber the customer's telephone number (any Swedish format)
```

```
 * @param problemDTO the problem description with the broken bike
 * @return the generated repair order ID
 * @throws NonExistentTelephoneNumberException if the customer is not found or
    phone format is invalid
 */
public String createRepairOrder(String telephoneNumber, ProblemDTO problemDTO)
    throws NonExistentTelephoneNumberException, InvalidTelephoneNumberException
    , FailedOperationException{
    try {
        CustomerDTO foundCustomer = searchCustomer(telephoneNumber);
        RepairOrder repairOrder = new RepairOrder(foundCustomer);
        repairOrder.updateProblem(problemDTO);
        repairOrderRegistry.save(repairOrder);
        return repairOrder.getId();
    } catch (ResourceNotFoundException | InvalidResourceURIException |
    CannotReadFileException | CannotWriteFileException | NoDatabaseException
    exception){
        throw new FailedOperationException("Fail to create repair order",
    exception);
    }
}
```

3.1.4 The `findRepairOrder` system operation

The `findRepairOrder` operation is responsible for retrieving an existing repair order associated with a specific customer. This operation is essential for continuing the workflow after a repair order has been created, as it allows subsequent actions such as diagnostics and task creation to be performed on the correct entity.

The operation is initiated from the View layer by providing a telephone number and is handled by the controller. The controller first normalizes the telephone number into E.164 format to ensure consistency. It then retrieves all repair order identifiers associated with the customer from the `RepairOrderRegistry`. From this set, the most recent repair order is selected and fetched from the registry. The retrieved repair order is converted into a `RepairOrderDTO` before being returned to the View.

If no repair orders exist for the given customer, an `IllegalArgumentException` is thrown. In the case of data inconsistency, where an identifier exists without a corresponding repair order, an `IllegalStateException` is raised.

This operation demonstrates how the system maintains continuity in the workflow by consistently retrieving the most recent repair order. The use of DTOs ensures that internal model data is not exposed directly to the View, preserving encapsulation. As a result, the system provides access to the current state of a repair order, enabling further processing in subsequent system operations.

The `findRepairOrder` operation retrieves the most recently created repair order associated with a customer.

Listing 6: Method for finding a repair order by a given telephone number `findRepairOrder` in `Controller` (shared by both technician and receptionist)

```java
private List<String> findRepairOrderIds(String telephoneNumber) throws InvalidTelephoneNumberException{
    String phoneNumberE164 = new TelephoneNumber(telephoneNumber).toE164();
    List<RepairOrder> repairOrders = repairOrderRegistry.findRepairOrdersByTelephoneNumber(phoneNumberE164);
    return repairOrders.stream()
            .sorted(Comparator.comparing(RepairOrder::getCreatedDate).reversed())
            .map(RepairOrder::getId)
            .collect(Collectors.toList());
}

/**
    * Finds the most recent repair order for the given telephone number.
    *
    * @param telephoneNumber the customer's telephone number (any Swedish format)
    * @return the most recent repair order as a DTO
    * @throws InvalidTelephoneNumberException if the phone number format is invalid
    * @throws IllegalArgumentException if no repair orders exist for the customer
    * @throws IllegalStateException if the repair order ID exists but the order cannot be found (data inconsistency)
    */
public RepairOrderDTO findRepairOrder(String telephoneNumber) throws InvalidTelephoneNumberException, IllegalArgumentException, IllegalStateException{
    List<String> repairOrderIds = findRepairOrderIds(telephoneNumber);
    if (repairOrderIds.isEmpty()) {
        throw new IllegalArgumentException("There is no repair order created for this customer.");
    }
    String id = repairOrderIds.get(0);
    RepairOrder repairOrder = repairOrderRegistry.findByRepairOrderId(id);
    if (repairOrder == null) {
        throw new IllegalStateException("Repair order not found for id: " + id);
    }
    return repairOrder.toDTO();
}
```

3.1.5 The `addDiagnosticResult` system operation

The `addDiagnosticResult` operation allows a technician to record the outcome of a diagnostic task for a specific repair order. This operation represents the inspection

phase of the workflow, where the condition of the bike is evaluated and documented.

The operation is initiated from the View layer after a repair order has been retrieved. The technician provides the repair order identifier, the name of the diagnostic task, and a `ResultDTO` describing the outcome. The controller retrieves the corresponding `RepairOrder` from the `RepairOrderRegistry` and delegates the update to the model layer. The `RepairOrder` then updates its internal `DiagnosticReport` by locating the specified task and applying the result. Once the update is complete, the modified repair order is saved back into the registry.

The responsibility for modifying diagnostic data is encapsulated within the model, ensuring that business rules remain centralized and protected from external layers. The controller acts only as a coordinator of the operation, without directly manipulating internal data structures.

As a result, the repair order is updated with new diagnostic information, which directly influences later stages of the workflow such as repair task creation and cost estimation.

The detailed `addDiagnosticResult` is implemented by adding a diagnostic result to a tied diagnostic task, which is identified using a repair order ID.

Listing 7: Add diagnostic result using `addDiagnosticResult` method in `TechnicianController`

```java
/**
 * Adds a diagnostic result to a diagnostic task identified by name within a repair order.
 *
 * @param repairOrderId the ID of the repair order
 * @param diagnosticTaskName the name (partial match) of the diagnostic task to update
 * @param result the result to apply to the diagnostic task
 * @throws IllegalStateException if the repair order is not found
 * @throws IllegalArgumentException if no diagnostic task matches the given name
 */
public void addDiagnosticResult(String repairOrderId, String diagnosticTaskName, ResultDTO result) throws IllegalStateException, IllegalArgumentException {
    RepairOrder repairOrder = repairOrderRegistry.findByRepairOrderId(repairOrderId);
    if (repairOrder == null) {
        throw new IllegalStateException("Repair order not found for id: " + repairOrderId);
    }
    repairOrder.updateDiagnosticResult(diagnosticTaskName, result);
    repairOrderRegistry.save(repairOrder);
}
```

3.1.6 The `createProposedRepairTask` system operation

The `createProposedRepairTask` operation allows a technician to define repair actions based on the results of the diagnostic phase. This operation represents the transition from inspection to repair planning, where identified issues are translated into concrete repair tasks.

The operation is initiated from the View layer after diagnostic results have been recorded. The technician provides a `RepairTaskDTO` containing a description of the repair, along with its associated cost and estimated duration. The controller retrieves the corresponding `RepairOrder` from the registry and delegates the responsibility of adding the task to the model layer. The `RepairOrder` then adds the new task to its `RepairTaskCollection`, ensuring that it becomes part of the overall repair plan. The updated repair order is subsequently stored in the registry.

The model layer manages the internal collection of repair tasks, ensuring consistency and preventing unauthorized access to internal data structures. This encapsulation allows the system to evolve without affecting higher-level components.

As a result, the repair order is extended with proposed tasks, forming the basis for customer approval and enabling the next stage of the workflow.

Listing 8: Internal method `proceedTechnicianDiagnosticActions` in `View` for proceeding receptionist preparation tasks

```java
private void proceedTechnicianDiagnosticActions(String telephoneNumber) throws NonExistentTelephoneNumberException, InvalidTelephoneNumberException {
    RepairOrderDTO repairOrderDTO = technicianController.findRepairOrder(telephoneNumber);
    printout("3. Technician - Requested repair order: ", repairOrderDTO);

    String repairOrderId = repairOrderDTO.id();
    String diagnosticTaskName = "Mechanical Safety Check";
    ResultDTO result = new ResultDTO(true, true, "Chain and gears should be replaced.");
    technicianController.addDiagnosticResult(repairOrderId, diagnosticTaskName, result);
    printout("4. Technician - Updated diagnostic task: ", diagnosticTaskName);

    RepairTaskDTO repairTask1 = new RepairTaskDTO("Replace Chain", "Removal of worn or stretched chain and installation of a new compatible e\u2011bike chain. Includes lubrication, tension adjustment, and drivetrain alignment check.", new Cost(500, "SEK"), 1);
    RepairTaskDTO repairTask2 = new RepairTaskDTO("Replace gears", "Replacement of worn or damaged rear cassette/freewheel or front chainrings. Includes removal of old components, installation of new gear set, indexing and tuning of derailleur(s).", new Cost(400, "SEK"), 2);
    technicianController.addRepairTask(repairOrderId, repairTask1);
    technicianController.addRepairTask(repairOrderId, repairTask2);
    printout("5. Technician - Created proposed repair tasks 01: ", repairTask1);
```

```
    printout("6. Technician - Created proposed repair tasks 02: ", repairTask2);
}
```

Detailed implementation of `addRepairTask` method attaches a specific repair task onto a repair order.

Listing 9: Add repair task using `addRepairTask` method in `TechnicianController`

```
/**
 * Adds a repair task to a repair order.
 *
 * @param repairOrderId the ID of the repair order
 * @param repairTask the repair task to add
 * @throws IllegalStateException if the repair order is not found
 */
public void addRepairTask(String repairOrderId, RepairTaskDTO repairTask) throws IllegalStateException {
    RepairOrder repairOrder = repairOrderRegistry.findByRepairOrderId(repairOrderId);
    if (repairOrder == null) {
        throw new IllegalStateException("Repair order not found for id: " + repairOrderId);
    }

    repairOrder.addRepairTask(repairTask);
    repairOrderRegistry.save(repairOrder);
}
```

3.1.7 The `acceptRepairOrder` and `rejectRepairOrder` system operation

The `acceptRepairOrder` operation is performed when a customer accepts the proposed repair tasks.

The operation is initiated from the View layer after the proposed repair tasks have been presented to the customer. The controller retrieves the corresponding `RepairOrder` from the registry and updates its state to `Accepted`. This state transition is handled within the model, ensuring that the lifecycle of the repair order remains consistent. After updating the state, the controller sends the repair order to the `Printer`, producing a formal output. The updated repair order is then saved in the registry.

As a result, the repair order is marked as accepted and confirmed, ready for execution while a formal record of the accepted repair is generated.

Listing 10: Internal method `proceedReceptionConfirmationActions` in `View` that handles receptionist confirmation tasks

```
private void proceedReceptionConfirmationActions(String telephoneNumber) throws NonExistentTelephoneNumberException, InvalidTelephoneNumberException {
    RepairOrderDTO repairOrderDTO = receptionistController.findRepairOrder(telephoneNumber);
```

```
    printout("7. Reception - found repair order: ", repairOrderDTO);

    printout("8. Reception - Accepted order");
    String repairOrderId = repairOrderDTO.id();
    receptionistController.acceptRepairOrder(repairOrderId);
}
```

Internal implementation of `acceptRepairOrder` marks the repair order as accepted.

Listing 11: Implementation of `acceptRepairOrder` in `ReceptionistController`

```
/**
 * Accepts a repair order, updates its state, saves it, and prints it.
 *
 * @param repairOrderId the ID of the repair order to accept
 * @throws IllegalStateException if the repair order is not found
 */
public void acceptRepairOrder(String repairOrderId) throws
    IllegalStateException{
    RepairOrder repairOrder = repairOrderRegistry.findByRepairOrderId(
    repairOrderId);
    if (repairOrder == null) {
        throw new IllegalStateException("Repair order not found for id: " +
    repairOrderId);
    }
    repairOrder.accept();
    repairOrderRegistry.save(repairOrder);
    printer.print(repairOrder);
}
```

The `rejectRepairOrder` operation is used when a customer declines the proposed repair tasks. This operation represents an alternative outcome in the workflow, where the repair process is not approved.

The operation is initiated from the View layer when the customer decides not to proceed. The controller retrieves the corresponding `RepairOrder` from the registry and updates its state to `Rejected`. This state change is performed within the model layer, ensuring that all lifecycle transitions are handled consistently. The updated repair order is then saved back into the registry.

By supporting both acceptance and rejection scenarios, the system provides a complete representation of the decision-making process within the repair workflow. The encapsulation of state changes within the model ensures that the system remains robust and maintainable.

As a result, the repair order is marked as rejected, and no further actions are performed on it.

Listing 12: Implementation of `rejectRepairOrder` in `ReceptionistController`

```
/**
 * Rejects a repair order and updates its state.
 *
```

```
 * @param repairOrderId the ID of the repair order to reject
 * @throws IllegalStateException if the repair order is not found
 */
public void rejectRepairOrder(String repairOrderId) throws
    IllegalStateException{
    RepairOrder repairOrder = repairOrderRegistry.findByRepairOrderId(
    repairOrderId);
    if (repairOrder == null) {
        throw new IllegalStateException("Repair order not found for id: " +
    repairOrderId);
    }
    repairOrder.reject();
    repairOrderRegistry.save(repairOrder);
}
```

3.2 Test results

Unit testing is conducted continuously throughout the development and implementation process and is treated as an integral part of each iteration. After the implementation of each system operation, unit tests are added for all newly created methods to ensure correctness and stability at an early stage.

In this chapter, one representative class from each architectural layer is selected and presented as an example. This approach keeps the discussion focused and avoids excessive code listings and repetitive explanations, while still demonstrating that all relevant test methods have been implemented and executed to validate the system behavior across layers.

3.2.1 Integration layer - `CustomerRegistry` class

In integration layer, classes are tested to produce the correct data proxy-based storage behavior. In this example, `CustomerRegistry` class is tested to ensure correct handling of `CustomerDTO` object storage behavior, which includes search, insert, remove and update operations.

Before each test case, a `CustomerRegistry` is created, and after each test case for cleaning up resources, the `registry` reference is set to null, forcing the program to cleanup the resource after use. The first test case involves testing to retrieve an existing customer by a telephone number as a key. Since the registry reads the data resource from a resource file with hard-coded test data, the input can be constructed aligned to the resource. It is known that the telephone number `+46701234567` correspond to the customer `Erik Lindqvist` with an email adress `erik.lindqvist@example.com`.

Listing 13: ...

```
class CustomerRegistryTest {

    private CustomerRegistry registry;

```

```java
    @BeforeEach
    void setUp() {
        registry = new CustomerRegistry();
    }

    @AfterEach
    void cleanUp() {
        registry = null;
    }

    @Test
    void testGetExistingCustomer() {
        assertDoesNotThrow(() -> {
            CustomerDTO customer = registry.find("+46701234567");
            assertNotNull(customer);
            assertEquals("Erik Lindqvist", customer.name());
            assertEquals("erik.lindqvist@example.com", customer.emailAddress())
    ;
        });
    }
}
```

A negative test is also built by entering a telephone number that does not correspond to any stored user. In this case, during intended program execution, a checked exception `NonExistentTelephoneNumberException` will be thrown.

Listing 14: ...

```java
@Test
void testGetNonExistingCustomer() {
    assertThrows(NonExistentTelephoneNumberException.class, () -> registry.find
    ("+46700000000"));
}
```

Saving and retrieving customer is another test case. The test case involves creating necessary information data with `CustomerDTO`, with bike details stored in `BikeDTO`, as well as saving it into the registry. Since saving method does not return values, verifying the correct behavior must be implemented implicitly, by retrieving the customer again and checking the values to the data being stored. In this case, the test case depends on the search test case which is assumed to produce correct behavior.

Listing 15: ...

```java
@Test
void testSaveAndRetrieveCustomer() {
    BikeDTO bike = new BikeDTO("CAKE", "Åik", "CK-2024-200");
    CustomerDTO newCustomer = new CustomerDTO("Ingrid Svensson", "+46709999999"
    , "ingrid.svensson@example.com", List.of(bike));
    registry.save(newCustomer);

```

```
    assertDoesNotThrow(() -> {
        CustomerDTO retrieved = registry.find("+46709999999");
        assertNotNull(retrieved);
        assertEquals("Ingrid Svensson", retrieved.name());
        assertEquals(1, retrieved.bikes().size());
        assertEquals("CAKE", retrieved.bikes().get(0).brand());
    });
}
```

3.2.2 Model layer - `RepairOrder` class

In model layer, test cases for `RepairOrder` is taken as an example. The test cases involves validating the behavior for common business operation attached to `RepairOrder` as an entity. It includes tests for creating a repair order, as well as attaching repair tasks to the repair order, as well as checking the correct total cost update and estimated complete date as repair tasks are added.

A basic test, `testBasic`, creates a repair order using the private helper method `createOrder`, which initializes a `RepairOrder` object with example customer and bike data, as well as a problem description. The test verifies that all essential fields are correctly initialized and are non-null.

Listing 16: ...

```
class RepairOrderTest {
    private RepairOrder createOrder() {
        BikeDTO bike = new BikeDTO("Brand", "Model", "SN");
        CustomerDTO customer = new CustomerDTO("Test", "+46701234567", "
    test@example.com", List.of(bike));
        RepairOrder order = new RepairOrder(customer);
        order.getProblem().setDescription("Flat tire");
        order.getProblem().setBrokenBike(bike);
        return order;
    }

    @Test
    void testBasic() {
        RepairOrder order = createOrder();
        assertNotNull(order.getCustomerDTO());
        assertEquals("Flat tire", order.getProblem().getDescription());
        assertNotNull(order.getCreatedDate());
        assertNotNull(order.getTotalCost());
        assertNotNull(order.getId());
        assertEquals(RepairOrderState.NewlyCreated, order.getRepairOrderState()
    );
    }
}
```

The test case `testAddRepairTask` verifies that adding a single repair task updates the internal state of the repair order. After adding a task, the test checks that the total cost reflects the added task and that an estimated completion date is calculated.

Listing 17: ...

```
@Test
void testAddRepairTask() {
    RepairOrder order = createOrder();
    order.getRepairTaskCollection().addRepairTask(new RepairTaskDTO("Fix chain"
    , "Replace chain", new Cost(500, "SEK"), 2));

    assertTrue(order.getTotalCost().getAmount() >= 500.0F);
    assertNotNull(order.getEstimatedCompleteDate());
}
```

The test case `testAddMultipleRepairTasks` extends this behavior by adding multiple repair tasks. It ensures that the total cost is accumulated correctly across all tasks, validating that the cost calculation logic handles multiple entries consistently.

Listing 18: ...

```
@Test
void testAddMultipleRepairTasks() {
    RepairOrder order = createOrder();
    order.getRepairTaskCollection().addRepairTask(new RepairTaskDTO("Task1", "
    Desc1", new Cost(300, "SEK"), 1));
    order.getRepairTaskCollection().addRepairTask(new RepairTaskDTO("Task2", "
    Desc2", new Cost(200, "SEK"), 3));

    assertTrue(order.getTotalCost().getAmount() >= 500.0F);
}
```

3.2.3 Controller layer - `ReceptionistController` class

For testing controller layer, `ReceptionistController` class is picked out as an example. The tests consists of checking the correct behavior of the system operation tied to receptionist role. The examples is tied to searching a customer by different telephone numbers.

Before each test case, a new instance of `ReceptionistController` is created using a `RegistryCreator`, which initializes the required registries, and a `Printer` dependency.

The first test case, `testSearchExistingCustomer`, verifies that searching for an existing customer using a valid telephone number returns the intended result. The test ensures that no exception is thrown and that the returned `CustomerDTO` contains expected data, such as the customer name.

Listing 19: ...

```
class ReceptionistControllerTest {
```

```
    private ReceptionistController controller;

    @BeforeEach
    void setUp() {
        RegistryCreator registryCreator = new RegistryCreator();
        controller = new ReceptionistController(registryCreator, new Printer())
    ;
    }

    @Test
    void testSearchExistingCustomer() {
        assertDoesNotThrow(() -> {
            CustomerDTO customer = controller.searchCustomer("0707654321");
            assertNotNull(customer);
            assertEquals("Astrid Johansson", customer.name());
        });
    }
}
```

The second test case, `testSearchNonExistingCustomer`, is a negative test case and validates the system behavior when a search is performed with a telephone number that does not correspond to any stored customer. In this case, the controller is expected to throw a `NonExistentTelephoneNumberException` during correct program execution.

Listing 20: ...

```
@Test
void testSearchNonExistingCustomer() {
    assertThrows(NonExistentTelephoneNumberException.class, () -> controller.
    searchCustomer("0700000000"));
}
```

Similar to the previous test case, the test case `testSearchInvalidPhoneNumber` ensures that a malformed telephone number causes the method to throw an `InvalidTelephoneNumberExcepti`

Listing 21: ...

```
@Test
void testSearchInvalidPhoneNumber() {
    assertThrows(InvalidTelephoneNumberException.class, () -> controller.
    searchCustomer("abc"));
}
```

3.2.4 Utility layer - `TelephoneNumber` class

Finally, the utility layer consists of common tools that are necessary and used in all layers of the program. Tests for `TelephoneNumber` is shown as example, where a provided telephone number is normalized and converted into E164 telephone number format, which acts as the canonical representation.

The test cases includes several positive cases with a valid telephone number in different forms, such as already E164 format in `testE164` and international format with two leading zeroes in `testInternationalPrefix`, as well as local prefix with a leading zero in `testLocalPrefix`. These cases must not throw any exception but a correct E164 conversion during correct program execution.

Listing 22: ...

```
@Test
void testE164() {
    assertDoesNotThrow(() -> {
        TelephoneNumber number = new TelephoneNumber("+46701234567");
        assertEquals("+46701234567", number.toE164());
    });
}

@Test
void testInternationalPrefix() {
    assertDoesNotThrow(() -> {
        TelephoneNumber number = new TelephoneNumber("0046701234567");
        assertEquals("+46701234567", number.toE164());
    });
}

@Test
void testLocalPrefix() {
    assertDoesNotThrow(() -> {
        TelephoneNumber number = new TelephoneNumber("0701234567");
        assertEquals("+46701234567", number.toE164());
    });
}
```

Additionally, negative tests are also conducted and includes malformed telephone number strings. The tests includes `testNonZeroPrefix` where no leading zero or plus sign is found as well as `testWrongPrefix` with three leading zeroes. In these cases, a checked exception `InvalidTelephoneNumberException` will be thrown.

Listing 23: ...

```
@Test
void testNonZeroPrefix() {
    assertThrows(InvalidTelephoneNumberException.class, () -> {
        TelephoneNumber number = new TelephoneNumber("1234567");
        number.toE164();
    });
}

@Test
void testWrongPrefix() {
    assertThrows(InvalidTelephoneNumberException.class, () -> {
```

```
        TelephoneNumber number = new TelephoneNumber("0001234567");
        number.toE164();
    });
}
```

3.2.5 Data layer - `JsonFileHandler` class

In data layer, the `JsonFileHandler` class is tested to verify the correct handling of JSON-based resource files. The tests includes both positive and negative test cases. Positive cases includes validation of successful reading of different resource files, while a negative test ensures an exception handling is thrown when attempting to access non-existing file.

For each test case, `JsonFileHandler` instance is created and initialized with a specific resource name, which is encapsulated and handled by the private internal method `createHandler`. After creating the file handler object, each read operation is executed. For testing reading customers (`testReadCustomers`), the method `readList` is used to deserialize the JSON content into a list of `CustomerDTO` objects. The test verifies that the resulting list is not empty, confirming that the data has been successfully loaded.

Listing 24: ...

```
private JsonFileHandler createHandler(String resourceName) {
    JsonFileHandler handler = new JsonFileHandler();
    handler.initDataHandler(resourceName);
    return handler;
}

@Test
void testReadCustomers() {
    JsonFileHandler handler = createHandler("customers");
    List<CustomerDTO> customers = handler.readList(CustomerDTO.class);
    assertFalse(customers.isEmpty());
}
```

Tests for reading diagnostic tasks resource file (`testReadDiagnosticTasks`) is similar to `testReadDiagnosticTasks` by reading and deserializing a JSON-based resource file, as well as checking the result as a non-empty list with non-empty fields such as name and cost. The test case is simplified and only checks the name and cost of first element as non-null, to ensure the list is correctly populated.

Listing 25: ...

```
@Test
void testReadDiagnosticTasks() {
    JsonFileHandler handler = createHandler("diagnosticTasks");
    List<DiagnosticTaskDTO> tasks = handler.readList(DiagnosticTaskDTO.class);
    assertFalse(tasks.isEmpty());
    assertNotNull(tasks.get(0).name());
```

```
    assertNotNull(tasks.get(0).cost());
}
```

Finally, a test case for reading a non-existent file is also written, `testReadNonExistentFile`. Correct program execution involves throwing an unchecked exception `ResourceNotFoundException` when attempting to read a non-existent file.

Listing 26: ...

```
@Test
void testReadNonExistentFile() {
    assertThrows(ResourceNotFoundException.class, () -> {
        createHandler("nonexistent");
    });
}
```

3.3 Result printouts

In this chapter, the JUnit test result and program execution result are illustrated.

3.3.1 Testing printout

The testing command `mvn compile test` produces the testing summary 27 of tests.

Listing 27: JUnit test execution summary

```
...
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 116, Failures: 0, Errors: 0, Skipped: 0
[INFO]
...
```

3.3.2 Sample program execution printout

The console output of a complete workflow run is shown, broken down by each step. The output is produced by executing the program using
`mvn compile exec:java -Dexec.mainClass="se.ebikerepair.startup.Main"`.

1. Search customer The receptionist searches for a customer by telephone number. The system normalizes the input to E.164 format and returns the matching customer with their registered bikes.

Listing 28: Printout from search customer system operation

```
1. Reception - Found customer:

================================
```

```
  Customer Information
================================
  Name:      Astrid Johansson
  Phone:     +46707654321
  Email:     astrid.johansson@example.com
  Bikes:
    - Brand: Monark, Model: E-Karin (S/N: MO-2024-010)
    - Brand: Skeppshult, Model: Elit (S/N: SK-2024-055)
================================
```

2. Create Repair Order The receptionist creates a new repair order for the customer's broken bike. The system generates a unique order ID and stores the order in the registry.

Listing 29: Printout from create repair order system operation

```
2. Reception - Created repair order with id:
f6ff76e9-bc5c-4a68-96f8-5e3223194795
```

3. Find Repair Order The technician retrieves the most recent repair order for the customer. The order is displayed with all details including the newly created problem description and the full set of unchecked diagnostic tasks.

Listing 30: Printout from find repair order system operation

```
3. Technician - Requested repair order:

================================
  Repair Order
================================
  Order ID:       f6ff76e9-bc5c-4a68-96f8-5e3223194795
  Status:         NewlyCreated
  Created:        Sat Apr 18 14:51:24 CEST 2026
  Est. Complete:  N/A
  Total Cost:     0.00 SEK
********************************
  Customer:
  - Name:      Astrid Johansson
  - Phone:     +46707654321
  - Email:     astrid.johansson@example.com
  - Bikes:
    - Brand: Monark, Model: E-Karin (S/N: MO-2024-010)
    - Brand: Skeppshult, Model: Elit (S/N: SK-2024-055)
********************************
  Problem:
    - Description: Broken bike chain
    - Broken bike: Skeppshult Elit (S/N: SK-2024-055)
********************************
  Diagnostic Report:
```

```
    Days: 0
    Cost: 0.00 SEK
    Description: This report contains pre-defined diagnostic tasks
    for e-bike inspection. Only tasks marked with [X] have been
    performed and contribute to the repair cost.
      --------------------------------
      [ ] 0.Basic Electrical System Check
            A general inspection of the e-bike's electrical
            components, including wiring, connectors, sensors, and
            display functionality. ...
      [ ] Cost: 300.00 SEK | Est. days: 1 | Result: have not checked yet
      --------------------------------
      [ ] 1.Battery Health & Capacity Test
            ...
      [ ] Cost: 425.00 SEK | Est. days: 2 | Result: have not checked yet
      --------------------------------
      [ ] 2.Motor Diagnostic & Noise Analysis
            ...
      [ ] Cost: 500.00 SEK | Est. days: 2 | Result: have not checked yet
      --------------------------------
      [ ] 3.Charger Functionality Test
            ...
      [ ] Cost: 200.00 SEK | Est. days: 1 | Result: have not checked yet
      --------------------------------
      [ ] 4.Drivetrain & Mechanical Safety Check
            Ensures gears, chain, brakes, bearings, and frame
            components work correctly with the electric assist system.
            Identifies mechanical issues affecting motor load.
      [ ] Cost: 275.00 SEK | Est. days: 1 | Result: have not checked yet
********************************
  Proposed Repair Tasks:
      (none)
================================
```

4. Add Diagnostic Result The technician records the result of the "Mechanical Safety Check" diagnostic task, marking it as checked and flagging it for repair.

Listing 31: Printout from add diagnostic result system operation

```
4. Technician - Updated diagnostic task:
[X] Drivetrain & Mechanical Safety Check -> Checked and plan to be repaired

********************************
  Diagnostic Report:
    Days: 1
    Cost: 275.00 SEK
    ...
      --------------------------------
      [X] 4.Drivetrain & Mechanical Safety Check
```

```
            Ensures gears, chain, brakes, bearings, and frame
            components work correctly with the electric assist system.
            Identifies mechanical issues affecting motor load.
      [TO BE REPAIRED] Cost: 275.00 SEK | Est. days: 1
            | Result: Chain and gears should be replaced.
********************************
```

5 and 6. Add Proposed Repair Tasks Based on the diagnostic findings, the technician proposes two repair tasks with cost and time estimates: replacing the chain and replacing the gears.

Listing 32: Printout from add proposed repair tasks

```
5. Technician - Created proposed repair tasks 01:

    --------------------------------
    [ ] Replace Chain
        Removal of worn or stretched chain and installation of a new
        compatible e-bike chain. Includes lubrication, tension
        adjustment, and drivetrain alignment check.
         Cost: 500.00 SEK | Est. days: 1


6. Technician - Created proposed repair tasks 02:
    --------------------------------
    [ ] Replace gears
        Replacement of worn or damaged rear cassette/freewheel or
        front chainrings. Includes removal of old components,
        installation of new gear set, indexing and tuning of
        derailleur(s).
         Cost: 400.00 SEK | Est. days: 2
```

7. Find Repair Order (Receptionist) The receptionist retrieves the updated repair order to review the diagnostic results and proposed repair tasks before presenting them to the customer. The total cost now reflects both diagnostic and repair costs, and the estimated completion date is calculated.

Listing 33: Printout from find repair order system operation

```
7. Reception - found repair order:

================================
  Repair Order
================================
  Order ID:       f6ff76e9-bc5c-4a68-96f8-5e3223194795
  Status:         NewlyCreated
  Created:        Sat Apr 18 14:51:24 CEST 2026
  Est. Complete:  Wed Apr 22 14:51:24 CEST 2026
```

```
  Total Cost:     1175.00 SEK
********************************
  Customer:
  - Name:      Astrid Johansson
  ...
********************************
  Problem:
    - Description: Broken bike chain
    - Broken bike: Skeppshult Elit (S/N: SK-2024-055)
********************************
  Diagnostic Report:
    Days: 1
    Cost: 275.00 SEK
    ...
      --------------------------------
      [X] 4.Drivetrain & Mechanical Safety Check
            Ensures gears, chain, brakes, bearings, and frame
            components work correctly with the electric assist system.
            Identifies mechanical issues affecting motor load.
      [TO BE REPAIRED] Cost: 275.00 SEK | Est. days: 1
            | Result: Chain and gears should be replaced.
********************************
  Proposed Repair Tasks:
    Days: 3
    Cost: 900.00 SEK
    --------------------------------
    [ ] Replace Chain
        ...
         Cost: 500.00 SEK | Est. days: 1
    --------------------------------
    [ ] Replace gears
        ...
         Cost: 400.00 SEK | Est. days: 2
================================
```

8. Accept Repair Order & Print The customer accepts the proposed repair. The system updates the order state to "Accepted" and sends it to the Printer, which outputs the final repair order.

Listing 34: Sample output from program execution

```
8. Reception - Accepted order

**** Printing repair order ****

================================
  Repair Order
================================
  Order ID:       f6ff76e9-bc5c-4a68-96f8-5e3223194795
```

```
  Status:         Accepted
  Created:        Sat Apr 18 14:51:24 CEST 2026
  Est. Complete:  Wed Apr 22 14:51:24 CEST 2026
  Total Cost:     1175.00 SEK
********************************
  Customer:
  - Name:      Astrid Johansson
  ...
********************************
  Problem:
    - Description: Broken bike chain
    - Broken bike: Skeppshult Elit (S/N: SK-2024-055)
********************************
  Diagnostic Report:
    Days: 1
    Cost: 275.00 SEK
    ...
      --------------------------------
      [X] 4.Drivetrain & Mechanical Safety Check
            Ensures gears, chain, brakes, bearings, and frame
            components work correctly with the electric assist system.
            Identifies mechanical issues affecting motor load.
      [TO BE REPAIRED] Cost: 275.00 SEK | Est. days: 1
            | Result: Chain and gears should be replaced.
********************************
  Proposed Repair Tasks:
    Days: 3
    Cost: 900.00 SEK
    --------------------------------
    [X] Replace Chain
        ...
         Cost: 500.00 SEK | Est. days: 1
    --------------------------------
    [X] Replace gears
        ...
         Cost: 400.00 SEK | Est. days: 2
================================
```

4 Discussion

This chapter evaluates the implemented program and its unit tests according to the assessment criteria for Seminar 3 of code implementation. The evaluation is based on concrete observations from the source code and unit tests developed for the *Repair Electric Bike* system.

4.1 Code Readability, Naming, and Conventions

The code is designed to be easy to understand, with packages, classes, methods, and variables having descriptive and self-explanatory names. For instance, classes such as `RepairOrder`, `CustomerRegistry`, and `ReceptionistController` clearly reflect their responsibilities in the system. Method names such as `searchCustomer`, `createRepairOrder`, and `addDiagnosticResult` describe their effects without requiring additional comments.

Java naming conventions described in Chapter 6 of the course literature are consistently followed. Classes use `PascalCase`, while methods and variables use `camelCase`. Constants are declared as `static final` fields and written in uppercase with underscores. This consistency improves readability and makes the codebase easy to navigate.

4.2 Code Size, Duplication, and Structure

No significant duplicated code was identified in the implementation. Common logic is split into shared methods or classes rather than being repeated across controllers or entities. For example, the lookup logic of `CustomerDTO` is centralized in `CustomerRegistry` instead of being duplicated in multiple controllers.

Methods and classes are kept reasonably small. Each method performs a single logical task, and large workflows are composed by delegating to smaller, well-named methods. This reduces complexity and makes control flow easier to follow. No methods were found to have unnecessarily long parameter lists. When multiple related values are required, they are grouped into DTOs such as `ProblemDTO` or `RepairTaskDTO`.

4.3 Use of Objects and Static Members

Primitive obsession is avoided by introducing domain-specific objects that group primitive types where appropriate. For example, monetary values are represented by the `Cost` class instead of scattered raw primitive numbers and strings, ensuring that amount and currency are always handled together. Similarly, complex input data is wrapped in DTOs rather than passed as multiple primitive parameters.

Static members are avoided except printout formats. Most data is instance-based, avoiding shared mutable state. The few static fields that exist are immutable constants, such as formatting strings for printout output, which is appropriate and does not introduce hidden dependencies.

4.4 MVC, Layers, Cohesion, and Coupling, and Encapsulation

The MVC and Layer patterns are applied consistently in the implementation. The View layer contains only presentation logic using hard-coded calls, the Controller layer coordinates system operations, and the Model layer encapsulates all business logic. The Integration layer handles data storage through in-memory registries.

High cohesion is achieved by assigning each class a single, clear responsibility. For example, `RepairOrder` manages repair-order state and behavior, while registry classes are responsible only for storing and retrieving objects. Low coupling is achieved by

ensuring that controllers do not manipulate internal model state directly, but instead invoke domain methods that enforce business rules.

Encapsulation is enforced throughout the system. Internal state is kept private and can only be modified through well-defined methods. Internal collections and entities are never exposed directly to external layers. Instead, immutable DTOs such as `RepairOrderDTO` are returned, ensuring that objects are kept intact and model invariants cannot be violated by the View or Controller layers.

4.5 Documentation and Comments

All public classes and methods include Javadoc comments, fulfilling the requirement of one comment per public declaration. The Javadoc descriptions explain purpose, parameters, return values, and potential error conditions at a high level. Inline comments inside methods are deliberately avoided, as clarity is achieved through expressive method and variable names rather than explanatory comments tied to implementation details.

4.6 Unit Testing Framework and Structure

Unit tests are implemented using JUnit, satisfying the requirement to use a testing framework. Test classes are placed in the same package structure as the system under test, but in a separate test source directory, which allows direct access to package-private methods while keeping test code separate from production code.

There is one test class per tested class for all controller, model, and integration classes that contain logic. Simple DTO classes that only provide constructors and getters are intentionally excluded from testing, in accordance with the seminar requirements.

4.7 Test Coverage, Parameter Selection, and Branch Testing

Tests are designed to be self-evaluating: a failing test produces an informative failure message, while passing tests produce no output. Both normal cases and error cases are covered. For example, customer and repair-order lookup operations are tested with both existing and non-existing identifiers to ensure correct behavior and meaningful error handling.

All public and package-private methods that contain logic are covered by tests, and different branches of conditional logic are exercised where applicable. Methods are tested with a variety of interesting parameter values rather than a single trivial case. The successful execution of the full test suite with zero failures demonstrates both functional correctness and adequate test coverage.

4.8 Explanation of Programming Best Practices

The report explicitly explains and motivates the use of programming best practices throughout the codebase. Object-oriented design principles such as high cohesion, low coupling, and strong encapsulation are not only applied in the implementation, but are also discussed and justified in the Introduction, Method, and Discussion chapters.

Best practices described in Chapter 6 of the course book are reflected in concrete implementation choices. For example, responsibilities are clearly separated between the View, Controller, Model, and Integration layers, avoiding mixed concerns and excessive dependencies. Primitive obsession is addressed by introducing domain-specific objects such as `Cost` and structured DTOs instead of passing multiple primitive values.

Coding conventions, including descriptive naming, consistent formatting, and limited use of static members, are explained in the Method chapter and evaluated in this Discussion. Refactoring decisions, such as removing duplicated code and splitting long methods, are also motivated as part of an ongoing effort to improve readability and maintainability. Together, these explanations demonstrate an understanding of how and why programming best practices are applied, rather than merely following them mechanically.

4.9 Reflection on Method, Result, and Discussion Relevance

The report provides a complete and coherent structure consisting of Introduction, Method, Result, and Discussion chapters, each serving a distinct and relevant purpose. The Method chapter explains how the implementation and testing were carried out, focusing on the development process rather than describing the solution itself. The Result chapter then presents the implemented system, including startup behavior, system operations, and sample execution output, demonstrating that the program is compilable, executable, and functionally correct.

This Discussion chapter builds on both the Method and Result chapters by evaluating the solution according to the official assessment criteria for Seminar 3, by analyzing the quality of the code and the tests with respect to readability, structure, object-oriented design principles, and testing adequacy. Concrete examples from the implemented system are used to support the evaluation and to demonstrate understanding of both programming and testing concepts.

Putting everything together, the report forms a consistent and relevant narrative that explains the solution development, the result, and evaluation of the result against the course's assessment criteria.

References

[1] Leif Lindbäck, *A First Course in Object-Oriented Development — A Hands-On Approach*, Course material for IV1350, KTH Royal Institute of Technology, January 14, 2026.

[2] Mo Wang, "ElectricBikeRepair — Source code for Seminar 3," 2026. [Online]. Available: `https://github.com/mo-mosverkstad/java-learning/tree/main/maven-projects/ElectricBikeRepair/`